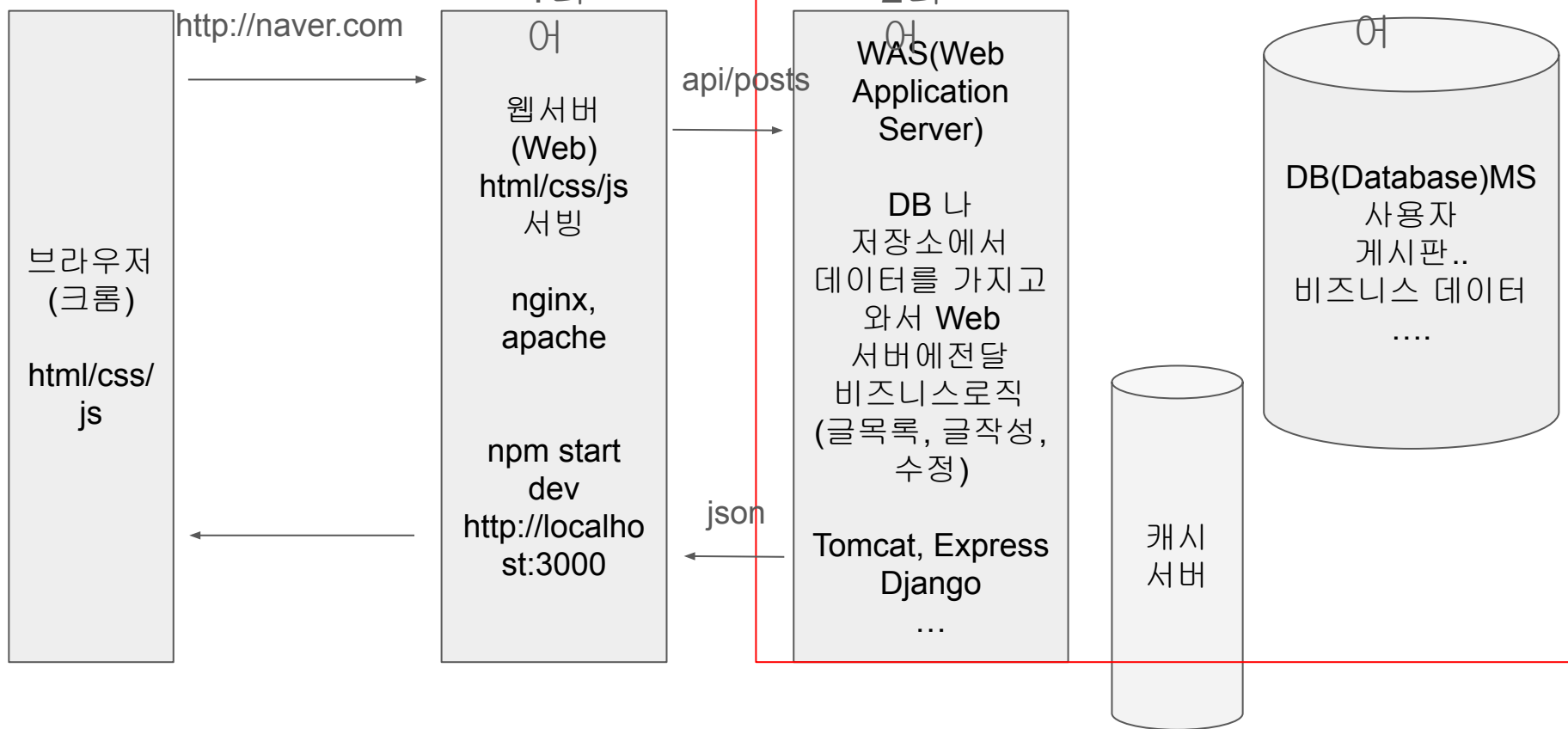


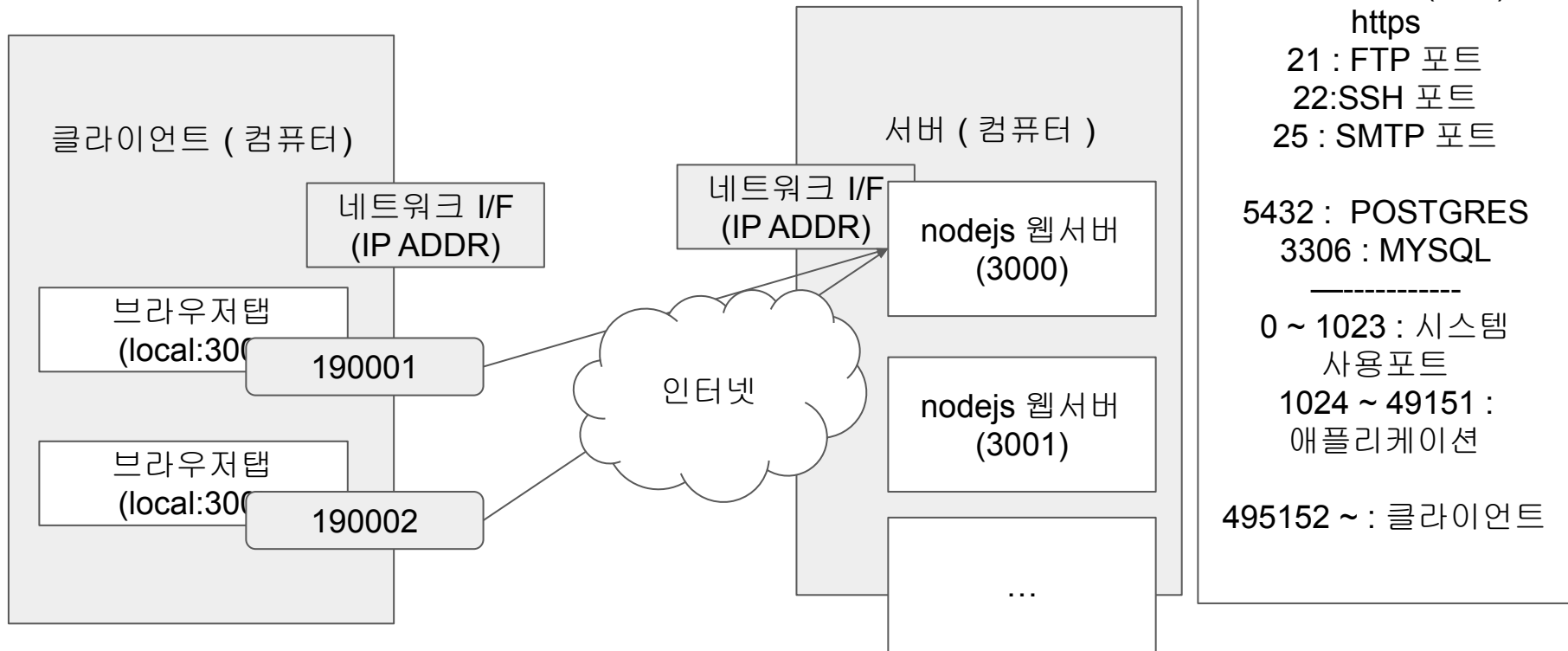
2026년 남부 백앤드 설명자료

2026.06

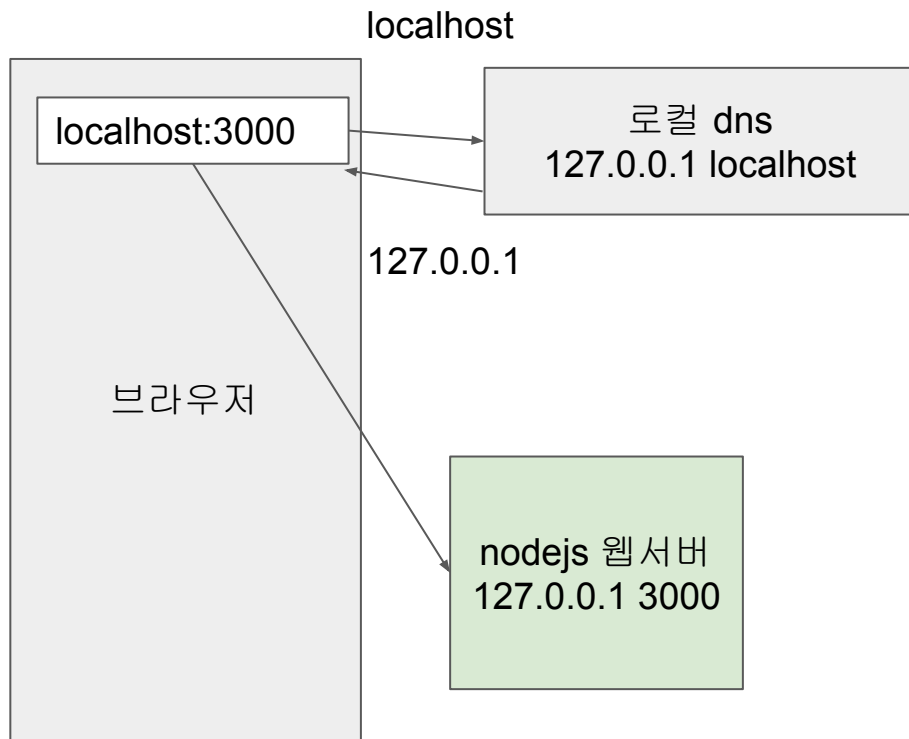
3-티어 아키텍처



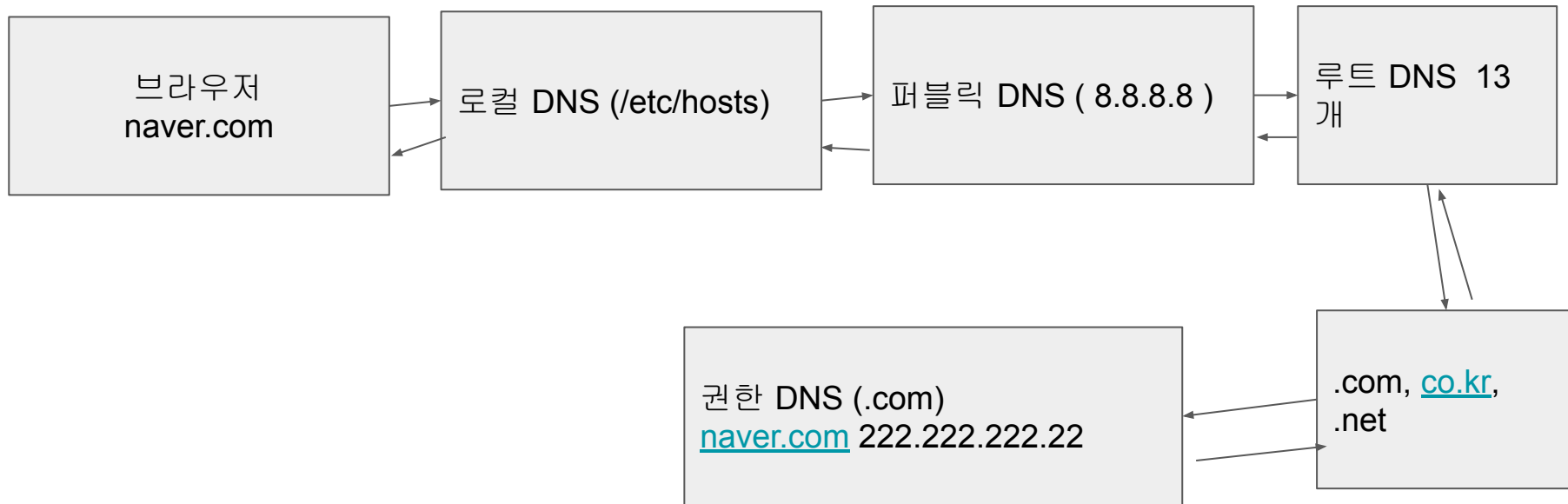
port



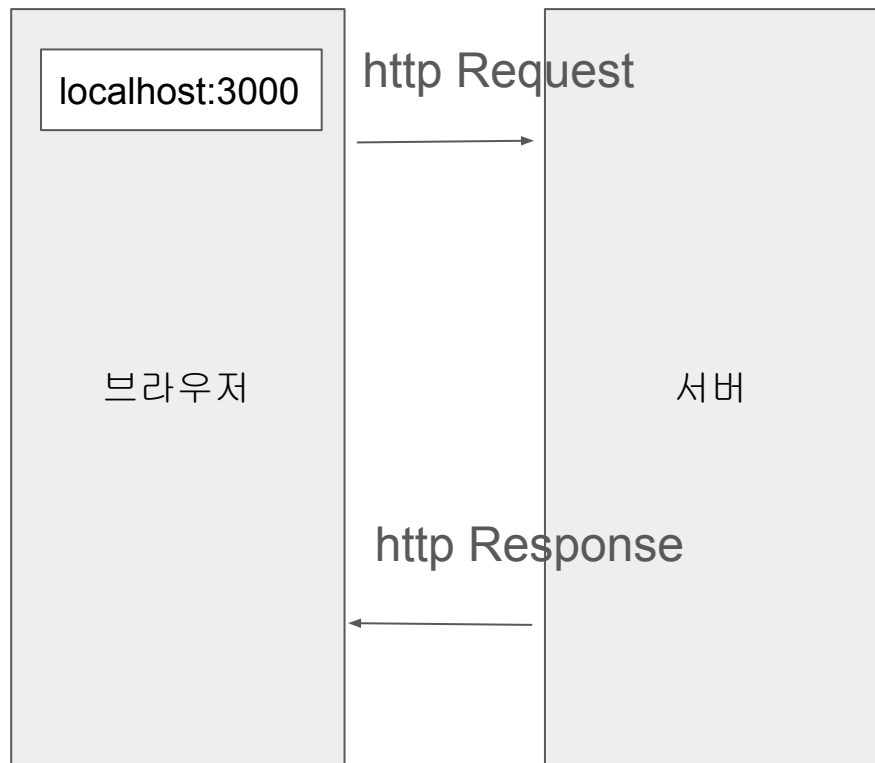
DNS 동작 방식 (localhost)



DNS



HTTP 요청과 응답



HTTP Request

항목	의미	예시
주소	무엇을 요청할지(자원) naver.com/payment	주소
헤더	부가정보(인증, 콘텐츠 등등)	인증정보
바디	데이터(게시글, 게글 제목, 작성이름)	

HTTP Response

항목	의미	예시
상태 코드	성공 실패 에러가났어..	200,
응답 헤더	부가정보, Content-Type	
응답 바디	서버에서 클라이언트에 보내는 응답	html, json

응답코드

- 200 : 요청이 성공
 - 200 : 요청이 성공
 - 201 : 요청이 성공 적이었고, 리소스가 생성되어 (게시글 작성)
- 300 :
 - 301: 요청한 리소스가 아주 다른 곳으로 갔어요
- 400 :
 - 400 : 요청이 잘못되었어요 (서버 A,B 클라이언트 A.C) ->
 - 401 : 인증오류 (아이디 비밀번호 가달라고, 자원에 권한이 없어요)
- 500 :
 - 500 : 서버에서 오류가 났어요 **try catch** -> 사용자가 자세한 오류를 볼 수 없게끔
 - 502 : 게이트웨이 또는 프록시 가 잘못되었어요.

파라미터 설명

<http://localhost:3000/list/3?search=자바스크립트&page=2&size=20>

http : 프로토콜

localhost: 도메인

3000 : 포트번호

list/3 -> path, uri

? -> get 파라미터 쿼리의 시작

search 키 , 자바스크립트는 값

& 연결자

HTTP 메소드 종류

1. GET : 서버에 리소스를 요청할 때 (게시글 목록을 주세요, 게시글 상세내용을 주세요)
2. POST : 서버에 데이터를 전송할 때 (게시글 작성)
3. PUT : 기존 리소스를 수정할때 (게시글 수정)
4. DELETE : 기존 리소스 삭제 (게시글 삭제)

query vs param

query -> <http://localhost/posts?author=김철수> : 검색 김철수 작성한 글만 보여주세요

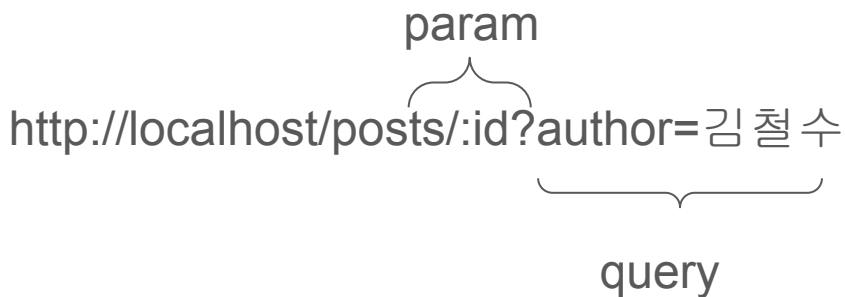
param -> <http://localhost/posts/1> : 특정글 1 개만 지정 해서 보여주세요 => 누구/무엇을 가르킬

<http://localhost/posts/1?author=김철수>

param

<http://localhost/posts/:id?author=김철수>

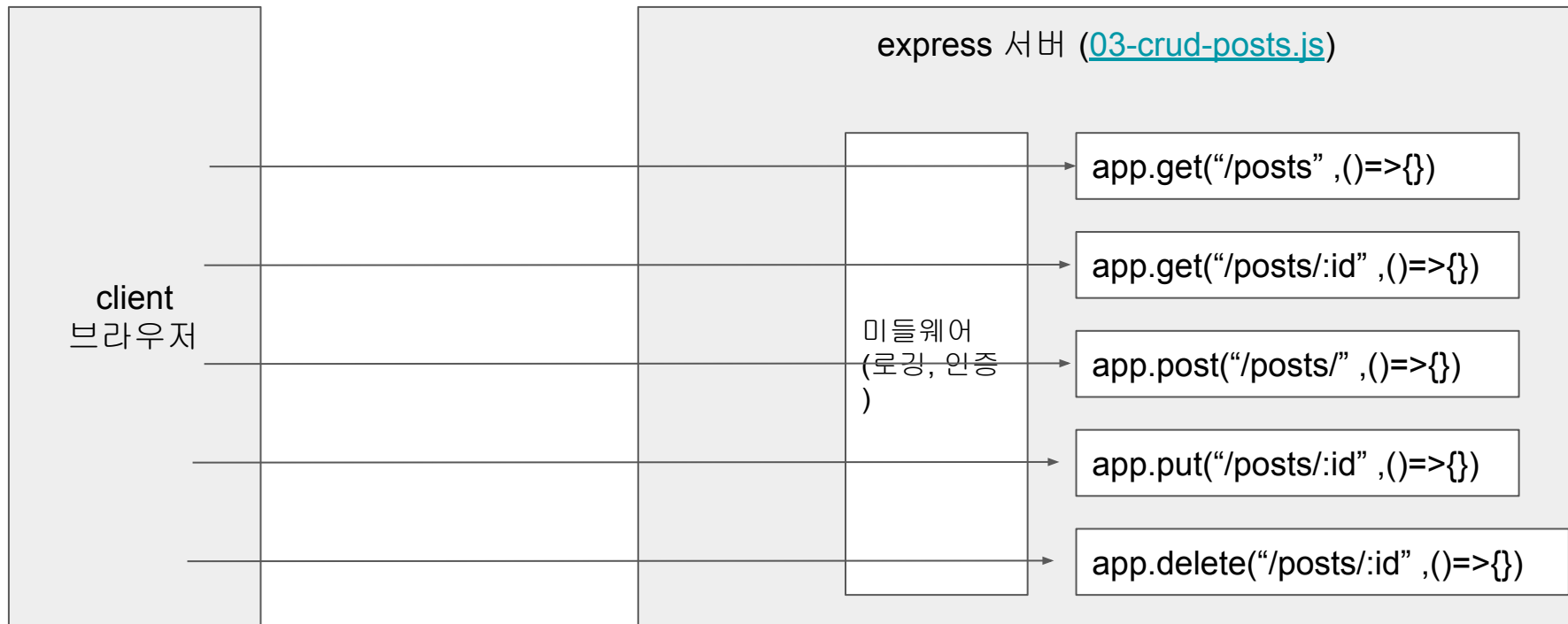
query



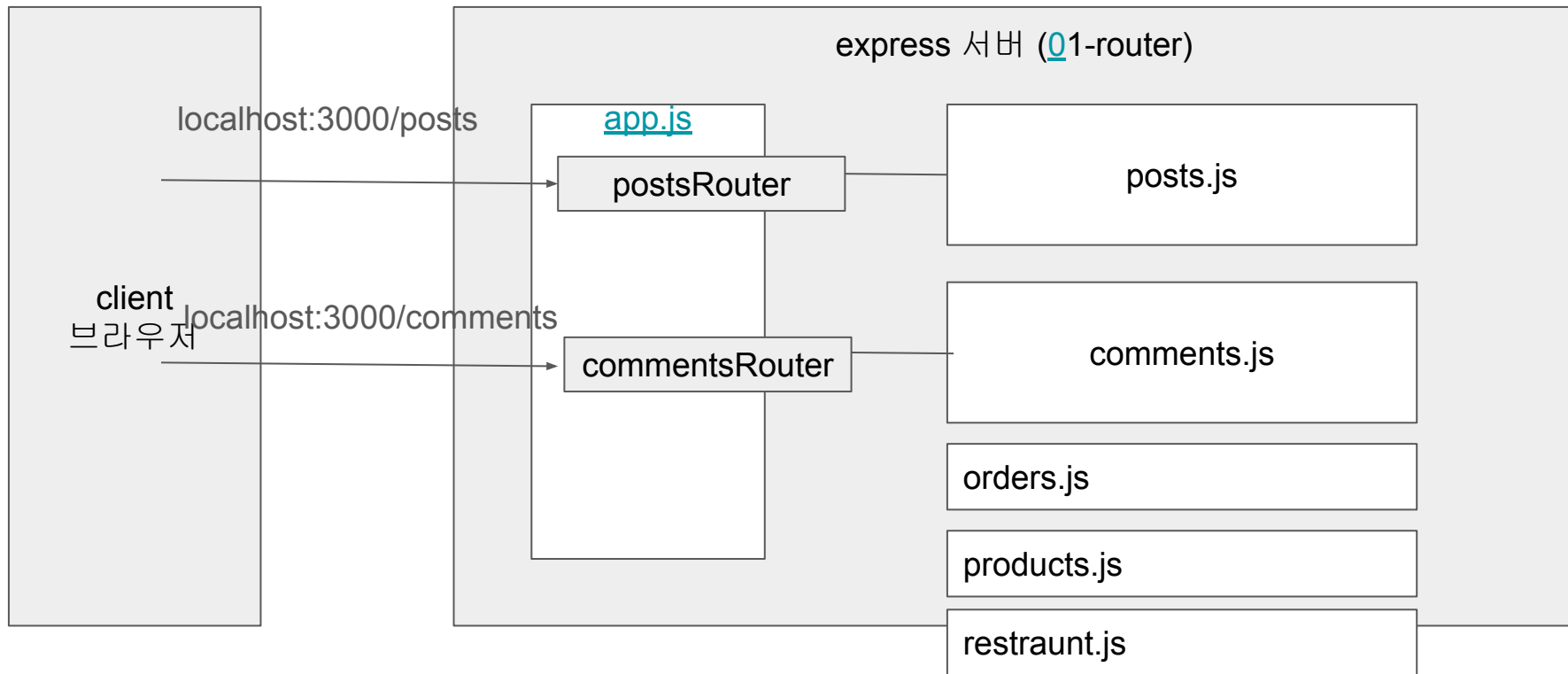
query vs param

상황	방식	예제
특정 리소스 1 개만	param	GET posts/1, posts/2 DELETE posts/1
검색 필터 정렬 페이징	query	?author=김... ?page=2, ?sort=date
없어도 동작	query	?author=
없으면 의미가 에러가남	param	GET posts/

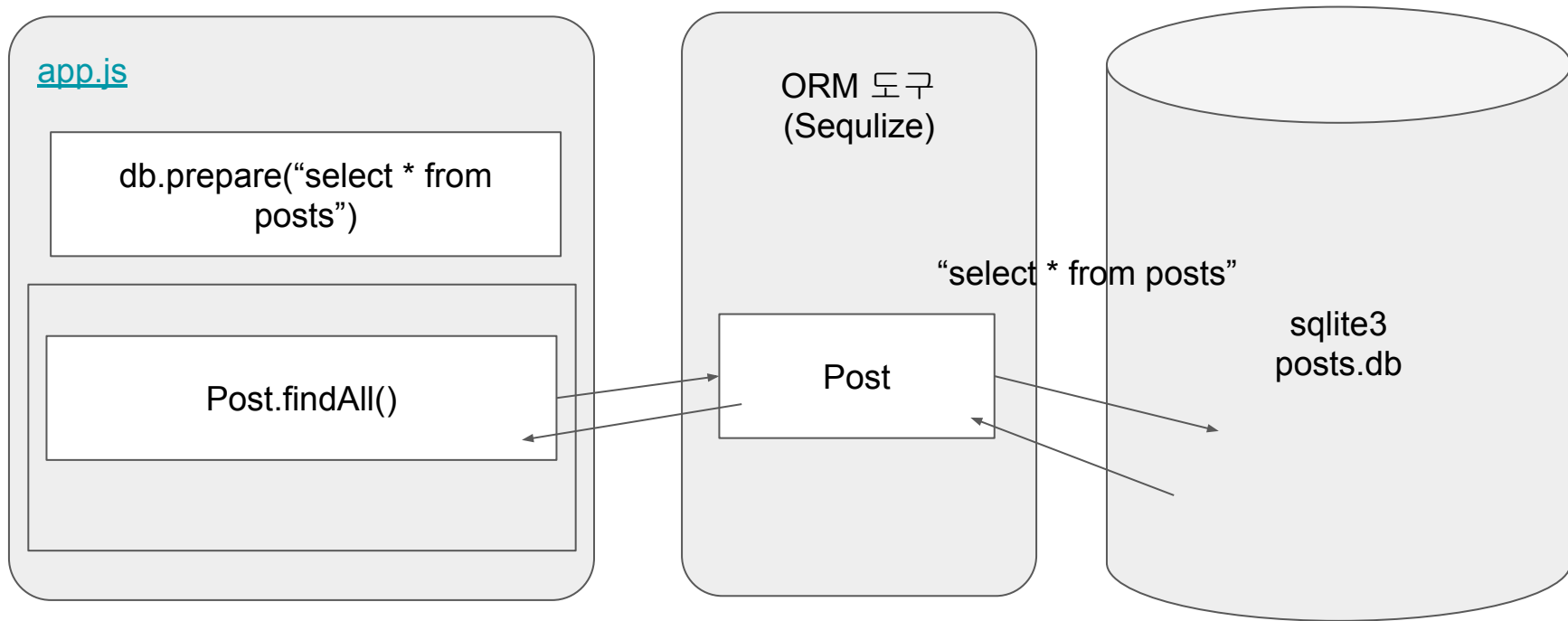
express middleware



express router



ORM 이란



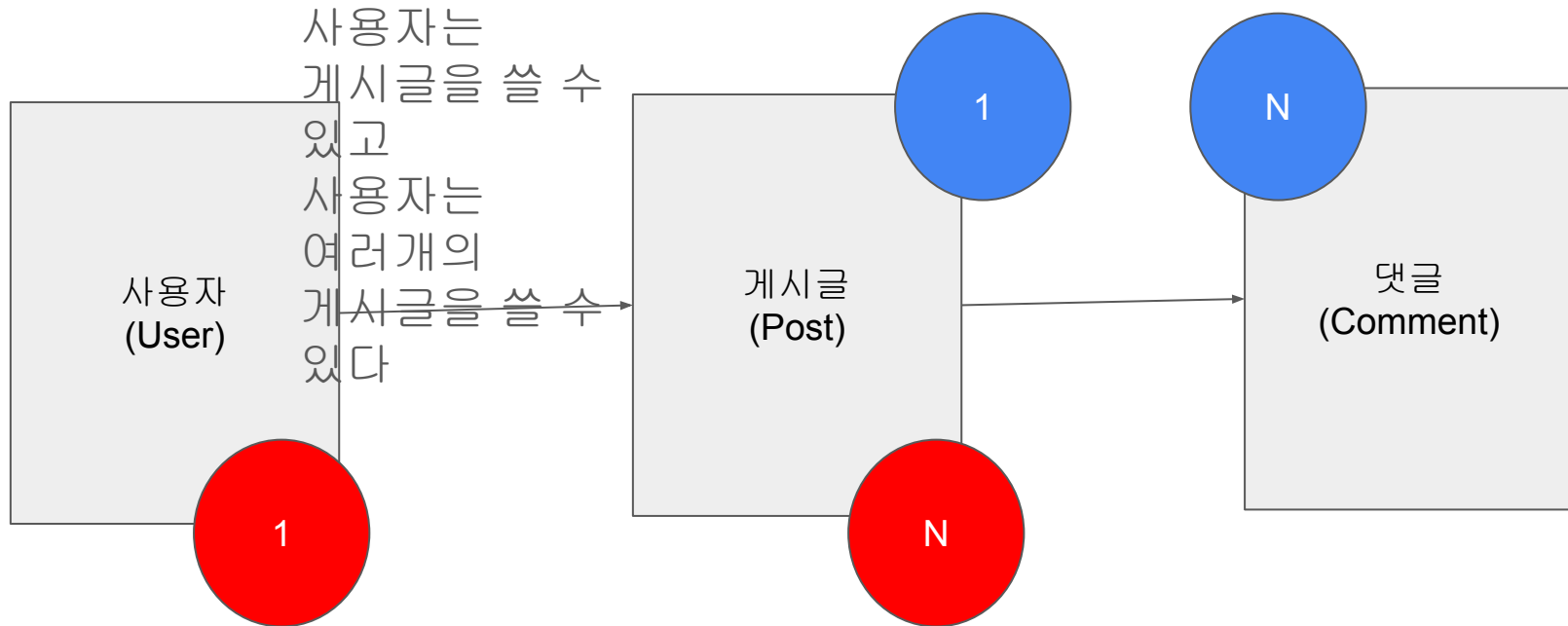
ORM 의 장점과 단점

- 생산성 향상 : **SQL** 을 안만드니까... 단) **SQL** 직접 만들어서 넣으면 **DB** 성능 (복잡한 **JOIN** 문이 들어가는 통계성 데이터라든지...
- 코드 일관성 :
- 보안성 증가 : **SQL Inject** 방지
- 유지보수성 증가 : 리팩토링에 강함
- **DB** 마이그레이션
- **DB** 를 개발 환경, 테스트 환경, 프로덕션 환경 별로 다르게 갈 수 있다.
 - 개발에서 **SQLITE**, 테스트에서는 **Postgres**, 프로덕션 오라클
- **SQL** 전부 바꾸지 않고 **DB**를 교체할 수 있다.

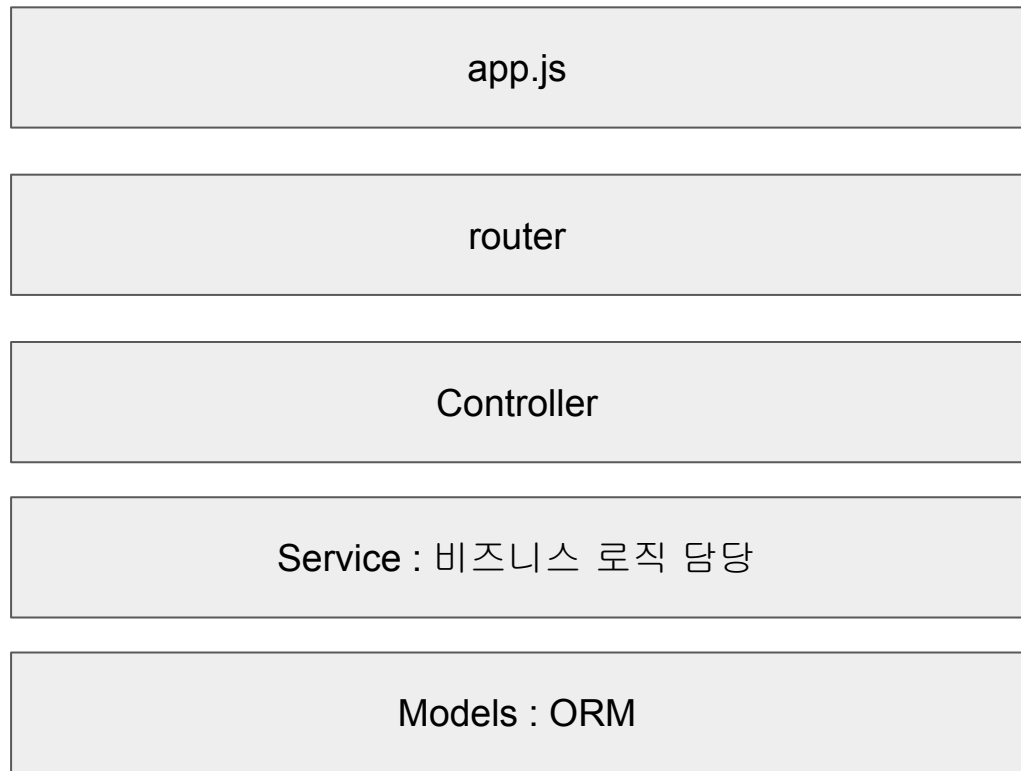
대표적인 ORM 도구

- [express.js](#) : **sequelize** → 요즘에는 대세
- typescript nestjs : **prisma**, typeorm
- 자바 : 스프링에서는 JPA, 하이버네이트
- python : django , flask + sqlalchemy
- ruby
-

사용자 - 게시물 - 댓글



Express-MVC



디렉토리 구조 및 파일구조

역할	Express	Spring	Nest
HTTP 처리/라우팅	routes/ controllers/	@Controller @RestController	@Controller
비즈니스로직	services/	@Service	@Inject @Service
데이터 접근	models/	@Repository	Repository(Primsa, T ypeORM)
도메인 엔티티		@Entity	@Entity
인입지점	app.js	@SpringBootApplication	@Module

Nest JS 는

- [Node.js](#) 기반 위에 동작하는 프레임워크, Express, Fastify 위 에 올라감
- Typescript 기반의 서버사이드 애플리케이션 프레임워크 입니다.
- 모듈화 중심의 아키텍처
- NestJS = Express + typescript + OOP + DI + 모듈화

NestJS 는 언제 써야 할까요

Expressjs	NestJS
코드가 자유롭고 유연하게 만들수 있음	정해진 틀 내에서 작성해야하고, 코드가 구조화되어 있어서 일관성 있는 코드 작성이 가능
유지보수 어렵습니다.	테스트 용이 , 확장성 있는 구조
규모가 커질 수록 복잡해집니다.	규모가 커져도 코드가 복잡하지 않습니다.
작은 프로젝트 프로토타입	대형 프로젝트에 적합한 모듈 구조

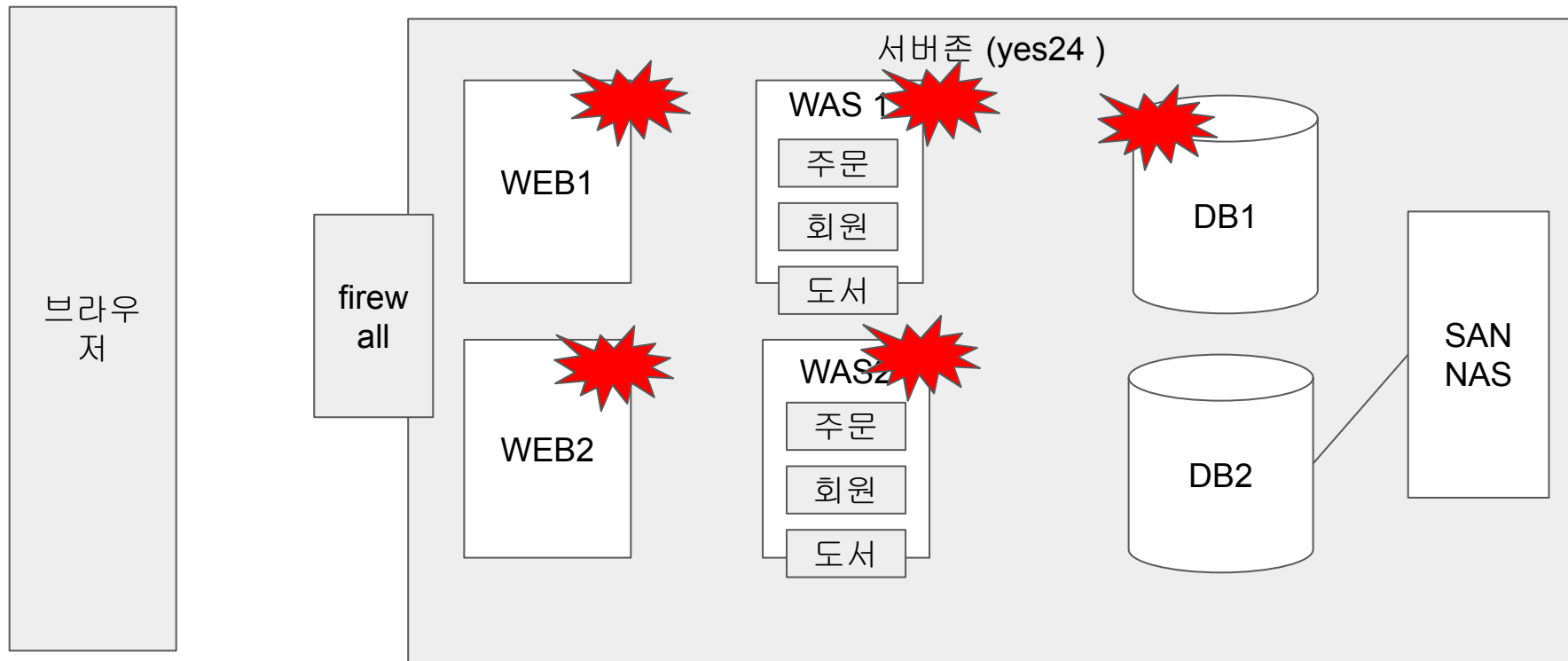
NestJS 의 핵심 개념

1. 모듈 : 기능단위로 묶는 컨테이너 개념, 모든 것이 모듈 단위로 구성되어 있어 **@Module**
2. 컨트롤러 : 요청(**HttpRequest**)처리하고, 응답(**HttpResponse**) 반환하는 역할만 수행합니다. **@Controller()**
3. 서비스 : 비즈니스 로직을 담당합니다. **DI(디펜던지 인젝션)** 를 통해 컨트롤러에 주입된다.
4. 데코레이터 : 클래스, 메서드 파라미터에 붙는 메타데이터 및 동작제어 **@Injectable, @Get @Post @Req**
5. 의존성 주입 (**DI**) - 객체 간의 의존 관계를 자동으로 해결해줌, **IoC** 컨테이너에서 알아서 해줍니다.

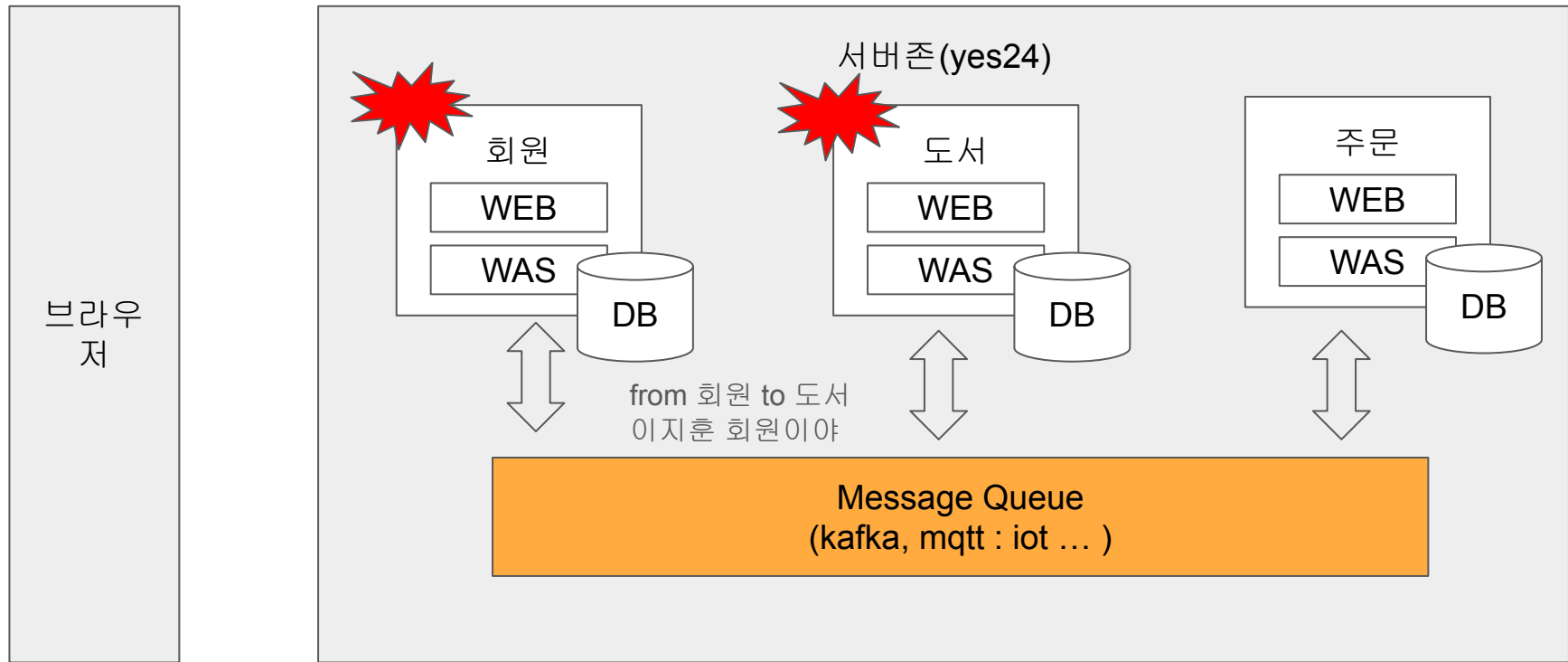
DI & IoC

- **Inversion of Control** (제어의 역전) : 객체 생성의 주체가 이전에는 개발자였는데. 이제는 개발자가 객체를 생성하지 않고, 프레임워크가 담당한다는 의미
- **NestJS**는 **IoC** 컨테이너가 클래스를 자동으로 인스턴스화 (객체로 만들고), **DI** 수행해줍니다.
- **DI** (의존성 주입) : 필요한 객체를 외부에서 자동으로 주입
- **IoC** : 객체 생성 및 주입을 프레임워크가 제어한 구조이다...

모놀리틱 아키텍처



MSA



페이징

1번 게시물
2번 게시물
3번 게시물
4번 게시물
5번 게시물
6번 게시물
7번 게시물
8번 게시물
9번 게시물
10번 게시물
11번 게시물
12번 게시물

1페이지

```
select * from posts  
limit 5 offset 0  
(page - 1) * limit = (1-1)* 5 = 0
```

2페이지

```
select * from posts  
limit 5 offset 5  
(page - 1) * limit = (2-1)* 5 = 5
```

3페이지

```
select * from posts  
limit 5 offset 10  
(page - 1) * limit = (3-1)* 5 = 10
```

prisma 명령어

npx prisma migrate dev	개발 중 스키마 변경을 DB에 적용하세요, + 마이그레이션파일생성
npx prisma generate	'schema.prisma' 기준으로 PrismaClient API 재생성 (findAll, findOne, update....)
npx prisma studio	브라우저에서 테이블 데이터 확인 및 수정
npx prisma format	schema.prisma를 들여쓰기 및 정렬 오타 도 잡고, 포매팅

prisma 기본 CRUD

작업내용	Prisma API	예시 코드
전체 조회	<code>findMany({where })</code>	
단건 조회	<code>findUnique({where})</code>	
생성	<code>create({data})</code>	
수정	<code>update({where, data})</code>	
삭제	<code>delete({where})</code>	
페이징	<code>skip (offset) take (limit)</code>	

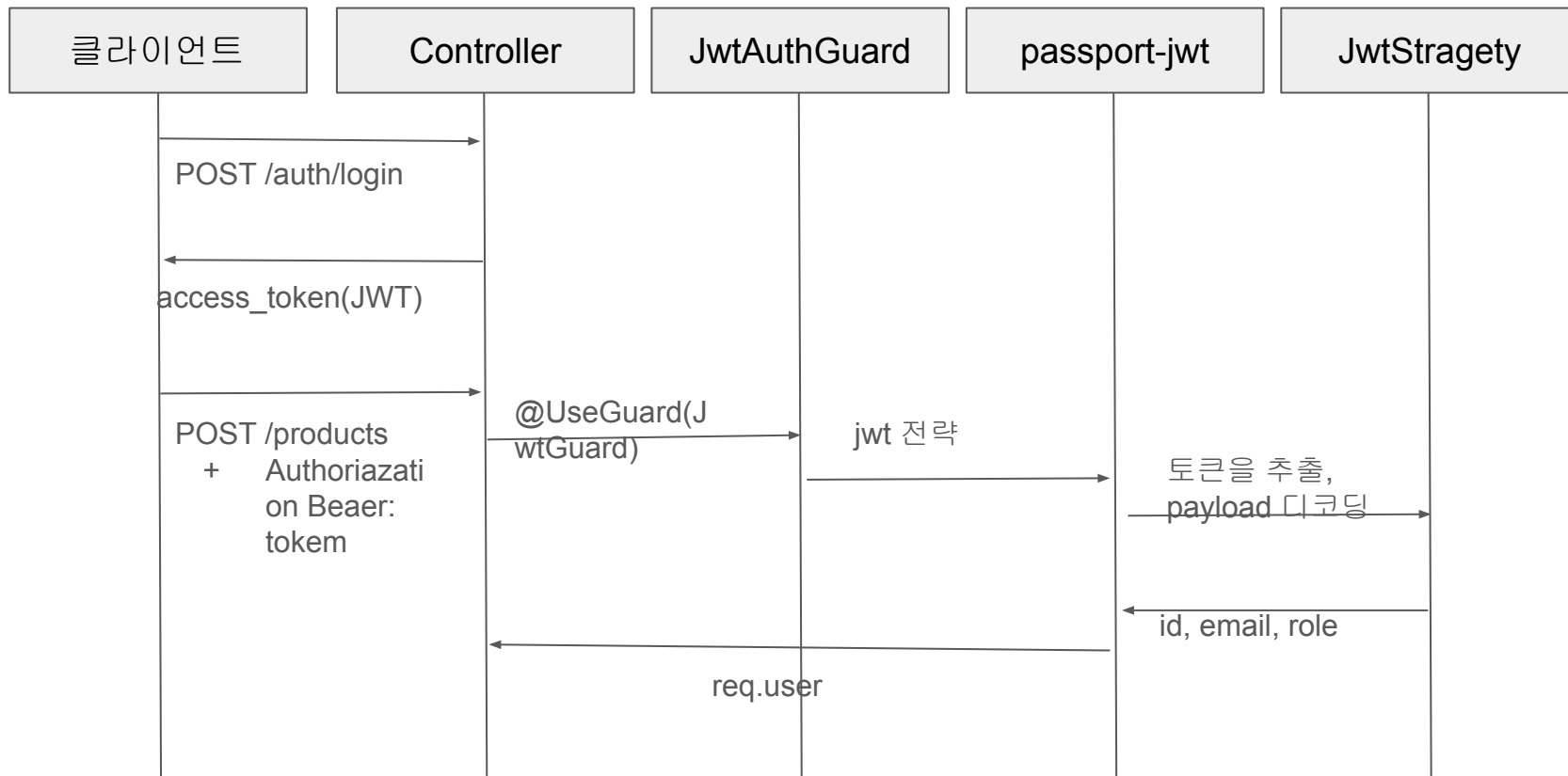
Prisma DataType

프리즈마 타입	의미	예시
Int	정수	id, price, stock
String	문자열	name, description
DateTime	날짜 및 시간	createdAt
Boolean	참/거짓	

자주 사용되는 속성(어노테이션)

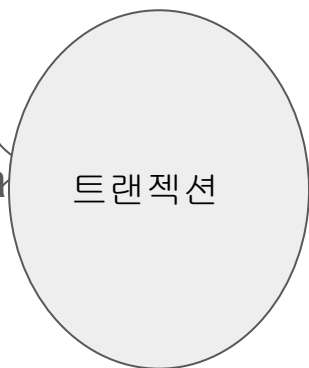
속성	의미	예시
@id	기본키 (PK)	
@default(autoincrement())	자동증가 1씩 INT	
@default(0)	값을 안넣으면 기본값이 0	Product stock
@default(now())	현재시각 넣어줘	Product CreatedAt
@unique	중복 불가 (유니크 제약)	Category name

NestJS 인증 전략



장바구니에서 주문테이블로

1. 장바구니에 담은 항목을 조회 (예를 들어 장바구니에 담은 제품이 10)
2. 장바구니에 다음 항목을들 모두 순회 (10번을 순회하면서)
 - a. 각각의 제품에 재고수량을 차감 (`product stock - cartItem.quantity`)
 - b. 각각의 장바구니 아이템의 정보를 `orderItem` 에 넣을 수 있도록 준비
3. 주문 정보(`Order`)를 생성 -> `OrderItem` 에서 `unitprice + -> order total`
4. 주문 상세 정보(`OrderItem`) 도 ~~도~~
5. 장바구니를 비워줘야 겠죠?



트랜잭션